

IHK – تحلیل امضای متدها و ترفندهای حل الگوریتم

بخش A: متدهایی که از صورت سؤال‌های واقعی (عکس‌هایی که فرستادی) آمده‌اند.
بخش B: الگوهای رایج دیگر که در آزمون‌های IHK زیاد تکرار می‌شوند (نمونه آموزشی).

بخش A – متدهای واقعی از امتحان‌های قبلی / عکس‌های تو

A1) setArray

FUNCTION setArray(laenge: integer) : arrayTyp integer

ورودی‌ها: یک عدد صحیح laenge (طول آرایه).

خروجی: یک آرایه جدید از نوع عدد صحیح (مثل int[]) با همین طول.

نوع مسئله: «ساختن و مقداردهی اولیه آرایه».

- گام ۱: یک آرایه جدید با اندازه laenge بساز: `new int[laenge]`.
- گام ۲: روی اندیس‌های ۰ تا laenge-1 حلقه بزن و برای هر خانه یک مقدار اولیه بگذار (معمولاً ۰ یا یک مقدار ثابت شروع).
- گام ۳: خود آرایه را برگردان.

ترفند: اگر تنها پارامتر «طول» است و خروجی آرایه، بدون فکر برو سراغ الگوی: `for + new array + مقداردهی + return array`.

A2) druckeTag

FUNCTION druckeTag(datum: Datum, tagesProtokoll: arrayTyp integer) : void

ورودی‌ها: یک تاریخ datum و یک آرایه عددی tagesProtokoll (مثلاً مقادیر در طول روز).

خروجی: هیچ (void) – فقط چاپ/نمایش انجام می‌دهد.

نوع مسئله: «گزارش / چاپ» روی آرایه.

- ابتدا تاریخ را با فرمت مناسب چاپ کن (روز/ماه/سال).
 - سپس روی آرایه پروتکل روز حلقه بزن (مثلاً برای هر ساعت).
 - برای هر خانه، اگر معنی‌دار است (مثلاً $\neq 0$) مقدار و ساعت متناظر را چاپ کن.
- ترفند: `void + اسم drucke/print => «فقط حلقه + چاپ»`، دنبال `return` جدی نگرد.

A3) erzeugeListe

erzeugeListe(persnr : int, zeiten : zweidim. Tabelle vom Typ int, jahr : int, monat : int)

ورودی‌ها: شماره شخص persnr، جدول دوبعدی zeiten، سال و ماه.

خروجی: یک لیست از رکوردهای زمان برای همان شخص در همان ماه/سال.

نوع مسئله: «فیلتر از ماتریس → لیست».

- دو حلقه: سطرها = روزها/افراد، ستون‌ها = بازه‌های زمانی.
 - برای هر خانه بررسی کن آیا رکورد متعلق به این persnr و این jahr/monat است.
 - اگر بله، یک آبجکت/رکورد جدید بساز (شامل تاریخ، ساعت، مقدار) و به لیست خروجی اضافه کن.
- ترفند: هر جا `int[][]` + کلمه «Liste» یا «erzeugeListe» باشد $\Rightarrow$ دو حلقه تو در تو + `if` شرط + افزودن به لیست.

A4) sucheTopseller

sucheTopseller(kriterium: Integer, vorgabewert: Integer) : String

ورودی‌ها: یک معیار (مثلاً نوع معیار ۱=تعداد، ۲=مبلغ) و یک مقدار حداقلی vorgabewert.

خروجی: یک رشته (مثلاً شماره کالا یا نام کالا) که «بهترین فروشنده» است.

نوع مسئله: «جستجوی حداکثر با شرط آستانه».

- لیست کالاها یا رکوردها را حلقه بزن.
 - برای هر رکورد مقدار معیار را حساب کن (بسته به kriterium).
 - اگر مقدار $\leq$ vorgabewert و بهتر از بهترین فعلی بود $\Rightarrow$ بهترین را به‌روزرسانی کن.
 - در پایان شناسه/نام آن رکورد را برگردان.
- ترفند: نام‌هایی مثل Top, Best, Max $\Rightarrow$ همیشه یک متغیر bestId + bestValue در حلقه نگه‌دار.

A5) erstelle_liste

erstelle_liste(stundensatz : Double)

ورودی‌ها: نرخ ساعتی stundensatz.

خروجی: غالباً یک لیست جدید یا تغییر روی ساختار موجود (void).

نوع مسئله: «محاسبه هزینه/قیمت برای همه رکوردها».

- روی همه کارمندان یا همه رکوردهای ساعت کار حلقه بزن.
 - برای هر رکورد: مبلغ = ساعات $\times$ نرخ ساعتی.
 - در صورت نیاز آبجکت جدید بساز و در لیست جدید ذخیره کن یا مستقیم چاپ کن.
- ترفند: اسم erstelle_liste $\Rightarrow$ ساختن مجموعه جدید از روی داده‌های موجود؛ همیشه حلقه روی مجموعه اصلی.

A6) onNewValue

onNewValue(sensor_id: Integer, value: Double, timestamp: Long)

ورودی‌ها: شناسه سنسور، مقدار جدید، زمان ثبت.

خروجی: void – رویداد را «پردازش» می‌کند.

نوع مسئله: «Event-Handler / ذخیره‌سازی مقدار جدید».

- رکورد جدید (sensor_id, value, timestamp) را در یک ساختار (لیست، آرایه، پایگاه‌داده) ذخیره کن.
 - ممکن است آخرین مقدار سنسور را در جدول سنسورها به‌روزرسانی کند.
 - ممکن است آستانه‌ها را بررسی کرده و در صورت تجاوز، آلارم یا flag بگذارد.
- ترفند: پیشوند on... در نام متد $\Rightarrow$ تقریباً همیشه Event-Handler بدون مقدار بازگشتی.

A7) maxPeriod

maxPeriod(sensor_id: Integer, mindestwert: Double) : Integer

ورودی‌ها: شناسه سنسور و حداقل مقدار مجاز.

خروجی: طول بزرگ‌ترین بازه‌ای که در آن مقدار سنسور همیشه $\leq$ حداقل بوده است (بر حسب تعداد نمونه یا زمان).

نوع مسئله: «بزرگ‌ترین سکانس متوالی با شرط».

- لیست مقادیر سنسور را به ترتیب زمان مرور کن.
- اگر مقدار $\leq$ حداقل: طول بازه جاری را +۱ کن؛ اگر نه، بازه جاری را صفر کن.
- در هر قدم، اگر طول بازه جاری $<$ طول بازه حداکثر بود، حداکثر را به‌روزرسانی کن.
- در پایان طول حداکثری را برگردان.

ترفند: کلماتی مثل «maximal zusammenhängend», «längste Periode» $\Rightarrow$ دو متغیر currentRun و maxRun.

A8) printList

PROZEDUR printList(datenliste: LISTE<DATENSATZ>, jahr: INTEGER)

ورودی‌ها: یک لیست از رکوردها + یک سال.

خروجی: void – فقط چاپ می‌کند.

نوع مسئله: «فیلتر ساده + چاپ».

- روی همهٔ عنصرهای datenliste حلقه بزن.
 - اگر فیلد سال رکورد برابر jahr بود، رکورد را با قالب مناسب چاپ کن.
- ترفند: PROZEDUR + لیست + یک شرط ساده $\Rightarrow$ یک if روی هر رکورد و چاپ.

A9) ermittlePrüfziffer

ermittlePrüfziffer(Integer[]) : Integer

ورودی: آرایه‌ای از ارقام یا اعداد صحیح (شماره حساب، بارکد و ...).

خروجی: یک رقم کنترلی (Prüfziffer).

نوع مسئله: «وزن‌دهی + جمع + Modulo».

- برای هر رقم یک وزن مشخص (مثلاً ۴، ۳، ۲، ...) در نظر بگیر.
 - در حلقه: `sum += ziffer[i] * gewicht[i]`.
 - در پایان `pruefziffer = sum % 10` یا فرمول شبیه آن را برگردان.
- ترفند: هرجا کلمه Prüfziffer دیدی، تقریباً همیشه الگوریتم: حلقه روی ارقام + mod آخر.

A10) getAppointment

getAppointment(month: Integer, day: Integer, id: Integer, slot: Integer) : boolean

ورودی‌ها: ماه، روز، شناسهٔ شخص/منبع، اسلات زمانی.

خروجی: true/false – ترمین وجود دارد یا نه (یا آزاد است یا نه، بسته به صورت سؤال).

نوع مسئله: «بررسی وجود در ساختار رزرو».

- در ساختار دادهٔ ترمین‌ها (لیست/جدول) جستجو کن.
- اگر رکوردی با همین ماه، روز، id و slot پیدا شد، بر اساس تعریف سؤال مقدار درست را برگردان (مثلاً true = رزرو شده).
- اگر تا آخر چیزی پیدا نشد، مقدار مخالف را برگردان.

ترفند: خروجی boolean + نام get/has/check + جستجوی خطی با return در وسط حلقه و یک return دوم در انتها.

A11) sort(raeume)

void sort(raeume: Array von Raum)

ورودی: آرایه‌ای از اشیای Raum.

خروجی: void – آرایه همان‌جا مرتب می‌شود.

نوع مسئله: «مرتب‌سازی ساده (Bubble/Selection)».

- دو حلقه: بیرونی از ۰ تا n-1، داخلی از i+1 تا n-1.
- اگر `raeume[j]` طبق معیار (مثلاً شمارهٔ اتاق یا ظرفیت) باید قبل از `raeume[i]` بیاید، آن‌ها را جابه‌جا کن.

ترفند: `sort(Array von ...)` در IHK = معمولاً مرتب‌سازی با دو حلقه و swap ساده.

A12) vergleicheMitarbeiter

vergleicheMitarbeiter(Mitarbeiter m1, Mitarbeiter m2) : Integer

ورودی: دو شیء Mitarbeiter.

خروجی: عدد منفی/صفر/مثبت برای کوچکتر/برابر/بزرگتر بودن.

نوع مسئله: «تابع مقایسه (Comparator)».

- یک فیلد مقایسه‌ای انتخاب می‌شود (مثلاً نام خانوادگی، حقوق، شماره پرسنلی).
- اگر m1 از m2 کوچکتر بود: `-1` return یا مقدار منفی.
- اگر برابر: `0` return.
- اگر بزرگتر: `1` return یا مقدار مثبت.

ترفند: دو پارامتر از یک نوع + خروجی `int` ⇒ این متد برای مرتب‌سازی با Comparator به کار می‌رود.

A13) sort(List<T>, Function<T>)

sort(List<T> liste, Function<T> f) : void

ورودی: لیستی عمومی از نوع T و یک تابع مقایسه f.

خروجی: void – لیست درجا مرتب می‌شود.

نوع مسئله: «مرتب‌سازی عمومی با Comparator».

- در حلقه‌ها به‌جای مقایسه مستقیم، از `f(a, b)` برای تصمیم گرفتن استفاده می‌شود.
- اگر `f(a, b) > 0` یعنی ترتیب اشتباه است و باید جابه‌جا شوند (بسته به تعریف).

ترفند: وجود `<T>` و `Function` ⇒ الگوریتم عمومی است، منطق مرتب‌سازی همیشه همان است، فقط مقایسه خارج‌سازی شده.

A14) Umrechnung

void Umrechnung(dezimalzahl : int)

ورودی: یک عدد ده‌دهی.

خروجی: void یا رشته نمایش عدد در مبنای دیگر (بسته به صورت سؤال).

نوع مسئله: «تبدیل مبنای ۱۰ به ۲».

- تا وقتی `n > 0`: باقیمانده تقسیم بر ۲ را ذخیره کن، `n = n / 2`.
 - باقیمانده‌ها را معکوس (از آخر به اول) کنار هم بگذار تا نمایش دودویی به‌دست آید.
- ترفند: `Umrechnung + dezimalzahl` ⇒ حلقه تقسیم و باقیمانده، سپس برعکس کردن رشته باقی‌مانده‌ها.

A15) statistik

statistik(verbrauch: int[][], limit: int) : Jahresstatistik

ورودی: ماتریس دوبعدی مصرف‌ها و یک حد آستانه limit.

خروجی: یک آبجکت Jahresstatistik شامل مین، مکس و لیست مصرف‌کننده‌های بالای limit.

نوع مسئله: «Min/Max + لیست مصرف‌کننده‌های بحرانی».

- دو حلقه روی ماتریس (سطرها = مصرف‌کننده، ستون‌ها = ماه‌ها).
- برای هر خانه: مین و مکس جهانی را به‌روزرسانی کن.
- اگر برای یک مصرف‌کننده حداقل یک مقدار `limit <` بود، او را در لیست limitVerbraucher اضافه کن.
- در پایان شیء Jahresstatistik را با این سه جزء برگردان.

ترفند: خروجی آبیجت آماری + [[]]int ⇒ تقریباً همیشه دو حلقه + به روزرسانی چند متغیر آماری.

A16) countVisitors

countVisitors(entry: ComeLeave) : Integer[][]

ورودی: ساختاری از رویدادهای ورود/خروج (ComeLeave).

خروجی: یک ماتریس دوبعدی Integer [][] که آمار بازدیدکننده‌ها را نشان می‌دهد.

نوع مسئله: «ساخت جدول شمارش از روی Log».

- ابعاد ماتریس خروجی مشخص می‌شود (مثلاً روز × ساعت).
 - روی هر رویداد حلقه: زمان را به روز/ساعت نگاشت کن.
 - بسته به نوع رویداد (آمدن/رفتن) خانه مربوطه را افزایش/کاهش بده یا به شکل موردنظر به روزرسانی کن.
- ترفند: Log از نوع «Come/Leave» + خروجی Integer [][] ⇒ تبدیل Log به جدول شمارش.

A17) sort(Tageskurs[])

sort(Tageskurs[] kurse, Function vergleiche) : void

ورودی: آرایه Tageskurs و تابع vergleiche.

خروجی: void – آرایه مرتب می‌شود.

نوع مسئله: «مرتب‌سازی با تابع مقایسه خارجی».

- مشابه sort قبلی است، اما داده‌ها قیمت‌های روزانه هستند.
 - معیار (مثلاً تاریخ، قیمت، تغییر درصدی) در تابع vergleiche پیاده‌سازی می‌شود.
- ترفند: sort + Function ⇒ یک اسکلت ثابت الگوریتم sort و فقط منطق مقایسه قابل تغییر است.

A18) notierungPlus

notierungPlus(Tageskurs[] kurse) : Integer

ورودی: آرایه قیمت‌های روزانه.

خروجی: تعداد روزهایی که قیمت نسبت به روز قبل افزایش داشته است.

نوع مسئله: «شمارش تغییر مثبت نسبت به عنصر قبلی».

- اگر طول آرایه ≥ 1 باشد، نتیجه ۰ است.
 - از اندیس ۱ تا آخر: اگر $kurse[i] > kurse[i-1]$ ⇒ شمارنده ++.
 - در پایان شمارنده را برگردان.
- ترفند: نام‌هایی با Plus, Steigerung, Anstieg روی آرایه زمانی ⇒ الگوریتم مقایسه با عنصر قبلی.

بخش B – الگوهای رایج دیگر (نمونه‌های آموزشی)

B1) berechneDurchschnitt

berechneDurchschnitt(werte: double[]) : double

ایده: میانگین آرایه را حساب می‌کند.

گام‌ها:

- اگر طول آرایه ۰ است ⇒ برگرداندن ۰.۰ یا رفتار توافق‌شده.
- حلقه روی آرایه، جمع مقادیر.

- تقسیم جمع بر تعداد عناصر و برگرداندن نتیجه.
- ترفند: اسم Durchschnitt/Mittelwert $\Rightarrow$ جمع + تقسیم بر n.

B2) berechneStatistik

berechneStatistik(werte: int[]) : Statistik

ایده: مین، مکس، میانگین و شاید تعداد را در یک آبجکت Statistik بسته‌بندی می‌کند.

گام‌ها:

- اگر لیست خالی است، مقادیر پیش‌فرض بده (0, 0, ...).
- متغیرهای min, max, sum را مقداردهی کن (مثلاً با عنصر اول).
- حلقه: برای هر مقدار min/max/sum را به‌روز کن.
- در پایان میانگین را حساب کن و شیء Statistik را برگردان.

B3) findeMitarbeiter

findeMitarbeiter(liste: List<Mitarbeiter>, id: int) : Mitarbeiter

ایده: در لیست کارمندان به دنبال کارمندی با id مشخص می‌گردد.

گام‌ها:

- روی لیست حلقه بزنی.
- اگر `m.getId() == id` $\Rightarrow$ همان m را برگردان.
- اگر تا آخر چیزی پیدا نشد $\Rightarrow$ null یا مقدار قراردادی برگردان.

B4) sucheArtikel

sucheArtikel(artikel: Artikel[], nr: int) : int

ایده: اندیس کالایی با شماره داده‌شده را در آرایه برمی‌گرداند.

الگو: جستجوی خطی با بازگشت اندیس یا -۱.

B5) filtereBestellungen

filtereBestellungen(bestellungen: List<Bestellung>, mindestwert: double) : List<Bestellung>

ایده: فقط سفارش‌هایی که مبلغ‌شان $\geq$ حداقل است در لیست جدید می‌آیند.

گام‌ها:

- لیست خروجی خالی بساز.
- روی لیست ورودی حلقه.
- $\text{if } (b.getBetrag() \geq \text{mindestwert}) \Rightarrow$ به لیست خروجی اضافه کن.

B6) zaehleAuszubildende

zaehleAuszubildende(personen: List<Person>) : int

ایده: تعداد کسانی که نقش‌شان «Azubi» است می‌شمارد.

الگو: شمارنده=0؛ حلقه؛ اگر شرط برقرار بود شمارنده++؛ در پایان شمارنده را برگردان.

B7) countFehler

countFehler(logs: LogEintrag[]) : int

ایده: تعداد خطاها یا سطح‌های خاصی از Log را می‌شمارد (مثلاً ERROR).
ساختار: دقیقاً مشابه شمارش Azubi.

B8) reservierePlatz

reservierePlatz(saal: Sitzplatz[][], reihe: int, platz: int) : boolean

ایده: اگر صندلی خالی است، رزرو می‌کند و true می‌دهد؛ اگر از قبل رزرو شده، false.
گام‌ها:

- به خانه `saal[reihe][platz]` نگاه کن.
- اگر وضعیت = frei $\Rightarrow$ وضعیت = true، belegt، در غیر این صورت false.

B9) storniereTermin

storniereTermin(terminliste: Termin[], id: int) : boolean

ایده: ترمینی با id مشخص را لغو می‌کند؛ اگر پیدا شد و لغو شد true، اگر نه false.
الگو: جستجو روی آرایه، در صورت یافتن حذف/علامت‌گذاری به‌عنوان لغوشده، سپس true؛ اگر پیدا نشد false.

B10) ladeKunden

ladeKunden(dateiname: String) : List<Kunde>

ایده: از فایل داده‌ها را می‌خواند و لیست مشتری‌ها را برمی‌گرداند (در شبه‌کد بیشتر روی حلقه و تبدیل تمرکز است تا روی IO واقعی).

B11) speichereKunden

speichereKunden(kunden: List<Kunde>, dateiname: String) : void

ایده: لیست مشتریان را سطر به سطر در فایل می‌نویسد؛ هر سطر یک رکورد فرمت‌شده.

B12) berechneGesamtPreis

berechneGesamtPreis(position: Position) : double

ایده: اگر Position زیرمجموعه‌ها دارد، قیمت کل = قیمت این + مجموع قیمت زیرمجموعه‌ها (الگوریتم بازگشتی).

B13) erhoehePreis

erhoehePreis(artikel: List<Artikel>, faktor: double) : void

ایده: روی همه کالاهای حلقه می‌زند و قیمت = قیمت $\times$ faktor می‌کند؛ تغییری روی لیست درجا.

B14) markiereUeberfaellig

markiereUeberfaellig(rechnungen: List<Rechnung>, heute: Datum) : void

ایده: هر فاکتوری که تاریخ سررسیدش قبل از امروز است، فیلد «überfällig» آن را true می‌کند.

B15) erstelleBericht

erstelleBericht(kurse: Tageskurs[]) : String

ایده: از روی آرایهٔ قیمت‌های روزانه یک متن چندخطی می‌سازد که هر خط شامل تاریخ و قیمت است؛ خروجی یک String بزرگ است.

B16) formatKundenliste

formatKundenliste(kunden: List<Kunde>) : String

ایده: برای هر مشتری یک خط متن می‌سازد (مثلاً "ID – Name – Ort") و همه را به هم می‌چسباند.

B17) istGueltigePLZ

istGueltigePLZ(plz: String) : boolean

ایده: بررسی می‌کند کدپستی درست است (مثلاً دقیقاً ۵ رقم و همه رقم). الگوریتم: طول را چک کن، سپس روی حروف حلقه و بررسی رقم بودن.

B18) pruefeLogin

pruefeLogin(benutzer: String, passwort: String) : boolean

ایده: در لیست کاربران می‌گردد؛ اگر ترکیب نام‌کاربری/رمز مطابق رکوردی بود true؛ در غیر این صورت false.